- **Meshtastic USB** siempre trabaja contra **PTY virtual**.
- **MeshCore USB** también trabaja contra **PTY virtual**.
- El broker **nunca** abre directamente el USB físico cuando el nodo es USB.
- Se usa **un watchdog único y genérico**.
- Se usan servicios `socat` separados para cada USB real.
- En cada escenario solo cambian:
  - `.env-usb`
  - qué servicios `socat` se habilitan
  - qué variante de `minibroker-emergencias.service` se usa
  - si APRS está activo o no

El broker soporta nodos `meshtastic_serial`, `meshtastic_tcp`, `meshcore_serial` y `meshcore_tcp`, y el routing entre nodos se define con `NODE_ROUTE_N`.
La pasarela APRS usa `APRS_GATE_ENABLED`, `APRS_CALL`, `KISS_HOST`, `KISS_PORT`, `MESHTASTIC_CH` y `APRS_INBOUND_NODE_ALIASES`; si esta lista va vacía, cae al nodo `PRIMARY`.

---

1. Convención fija para todos los despliegues

1.1 Puertos físicos y virtuales

Meshtastic USB

- USB real 1: `/dev/ttyUSB0`
- PTY virtual 1: `/dev/ttyV0`
- USB real 2: `/dev/ttyUSB1`
- PTY virtual 2: `/dev/ttyV1`

MeshCore USB

- USB real 1: `/dev/ttyUSB2`
- PTY virtual 1: `/dev/ttyMC0`
- USB real 2: `/dev/ttyUSB3`
- PTY virtual 2: `/dev/ttyMC1`

Esto evita solapes entre Meshtastic y MeshCore. Si en tu equipo cambian los `ttyUSBx`, lo ideal es fijarlos con `udev`, pero para las pruebas funcionales esta convención deja todo claro.

2. Servicios `socat` completos

2.1 Meshtastic USB 1

`/etc/systemd/system/meshtastic-socat.service`

```
[Unit]
Description=Socat bridge Meshtastic USB1 -> PTY
After=local-fs.target
Wants=local-fs.target
StartLimitIntervalSec=0

[Service]
Type=simple
User=root
ExecStart=/usr/bin/socat -d -d -d \
PTY,link=/dev/ttyV0,raw,echo=0,waitslave,mode=660,group=dialout \
FILE:/dev/ttyUSB0,raw,echo=0,b115200,nonblock
Restart=always
RestartSec=2
KillMode=process
TimeoutStopSec=5
ExecStartPre=/bin/sleep 2

[Install]
WantedBy=multi-user.target
```

2.2 Meshtastic USB 2

`/etc/systemd/system/meshtastic-socat-2.service`

```
[Unit]
Description=Socat bridge Meshtastic USB2 -> PTY
After=local-fs.target
Wants=local-fs.target
StartLimitIntervalSec=0

[Service]
Type=simple
User=root
ExecStart=/usr/bin/socat -d -d -d \
PTY,link=/dev/ttyV1,raw,echo=0,waitslave,mode=660,group=dialout \
```

```
FILE:/dev/ttyUSB1,raw,echo=0,b115200,nonblock
Restart=always
RestartSec=2
KillMode=process
TimeoutStopSec=5
ExecStartPre=/bin/sleep 2

[Install]
WantedBy=multi-user.target
```

2.3 MeshCore USB 1

`/etc/systemd/system/meshcore-socat.service`

```
[Unit]
Description=Socat bridge MeshCore USB1 -> PTY
After=local-fs.target
Wants=local-fs.target
StartLimitIntervalSec=0

[Service]
Type=simple
User=root
ExecStart=/usr/bin/socat -d -d -d \
PTY,link=/dev/ttyMC0,raw,echo=0,waitslave,mode=660,group=dialout \
FILE:/dev/ttyUSB2,raw,echo=0,b115200,nonblock
Restart=always
RestartSec=2
KillMode=process
TimeoutStopSec=5
ExecStartPre=/bin/sleep 2

[Install]
WantedBy=multi-user.target
```

2.4 MeshCore USB 2

`/etc/systemd/system/meshcore-socat-2.service`

```
[Unit]
Description=Socat bridge MeshCore USB2 -> PTY
After=local-fs.target
Wants=local-fs.target
StartLimitIntervalSec=0

[Service]
```

```
Type=simple
User=root
ExecStart=/usr/bin/socat -d -d -d \
PTY,link=/dev/ttyMC1,raw,echo=0,waitslave,mode=660,group=dialout \
FILE:/dev/ttyUSB3,raw,echo=0,b115200,nonblock
Restart=always
RestartSec=2
KillMode=process
TimeoutStopSec=5
ExecStartPre=/bin/sleep 2

[Install]
WantedBy=multi-user.target
```

---

3. Watchdog único endurecido para todos los escenarios

El watchdog actual está orientado a Meshtastic USB y además supervisa MeshCore solo por puerto físico.
Para homogeneizar todo, hay que sustituirlo por uno único que vigile:

- USB real y PTY de Meshtastic
- USB real y PTY de MeshCore
- servicios `socat`
- broker
- Direwolf si `APRS_GATE_ENABLED=1`
- broker zombie por `broker_status.json`
- silencio extendido de nodos MeshCore

`/usr/local/bin/minibroker_usb_watchdog.sh`

```
#!/usr/bin/env bash
set -u

BROKER_SERVICE="minibroker-emergencias.service"
DIREWOLF_SERVICE="direwolf.service"
STATUS_FILE="${STATUS_FILE:-/opt/minibroker/data/broker_status.json}"

MESHTASTIC_ENABLE="${MESHTASTIC_ENABLE:-0}"
MESHTASTIC_USB_REAL="${MESHTASTIC_USB_REAL:-/dev/ttyUSB0}"
MESHTASTIC_USB_VIRTUAL="${MESHTASTIC_USB_VIRTUAL:-/dev/ttyV0}"
MESHTASTIC_SOCAT_SERVICE="${MESHTASTIC_SOCAT_SERVICE:-meshtastic-socat.
service}"
```

```
MESHTASTIC_ENABLE_2="${MESHTASTIC_ENABLE_2:-0}"
MESHTASTIC_USB_REAL_2="${MESHTASTIC_USB_REAL_2:-/dev/ttyUSB1}"
MESHTASTIC_USB_VIRTUAL_2="${MESHTASTIC_USB_VIRTUAL_2:-/dev/ttyV1}"
MESHTASTIC_SOCAT_SERVICE_2="${MESHTASTIC_SOCAT_SERVICE_2:-meshtastic-s
ocat-2.service}"

MESHCORE_ENABLE="${MESHCORE_ENABLE:-0}"
MESHCORE_SERIAL_PORT="${MESHCORE_SERIAL_PORT:-/dev/ttyUSB2}"
MESHCORE_VIRTUAL_PORT="${MESHCORE_VIRTUAL_PORT:-/dev/ttyMC0}"
MESHCORE_SOCAT_SERVICE="${MESHCORE_SOCAT_SERVICE:-meshcore-socat.serv
ice}"

MESHCORE_ENABLE_2="${MESHCORE_ENABLE_2:-0}"
MESHCORE_SERIAL_PORT_2="${MESHCORE_SERIAL_PORT_2:-/dev/ttyUSB3}"
MESHCORE_VIRTUAL_PORT_2="${MESHCORE_VIRTUAL_PORT_2:-/dev/ttyMC1}"
MESHCORE_SOCAT_SERVICE_2="${MESHCORE_SOCAT_SERVICE_2:-meshcore-socat
-2.service}"

APRS_GATE_ENABLED="${APRS_GATE_ENABLED:-0}"
LOG_TAG="minibroker-usb-watchdog"
need_broker_restart=0

log() {
  /usr/bin/logger -t "$LOG_TAG" "$1"
  printf '%s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$1"
}

restart_broker_now() {
  log "Reiniciando broker: $BROKER_SERVICE"
  /usr/bin/systemctl restart "$BROKER_SERVICE"
  exit 0
}

ensure_service_active() {
  local svc="$1"
  if ! /usr/bin/systemctl is-active --quiet "$svc"; then
      log "Servicio inactivo. Reiniciando $svc"
      /usr/bin/systemctl restart "$svc"
      sleep 3
      need_broker_restart=1
  fi
}

recover_pty() {
  local svc="$1"
  local pty="$2"

  log "PTY ausente: $pty. Reiniciando $svc"
```

```
  /usr/bin/systemctl restart "$svc"

  local recovered=0
  for i in {1..10}; do
      sleep 1
      if [ -e "$pty" ]; then
          log "PTY recuperado: $pty tras ${i}s"
          recovered=1
          break
      fi
  done

  if [ "$recovered" -eq 0 ]; then
      log "CRÍTICO: PTY no apareció tras 10s: $pty"
      exit 1
  fi

  need_broker_restart=1
}

check_usb_pair() {
  local enabled="$1"
  local real_dev="$2"
  local virtual_dev="$3"
  local svc="$4"
  local label="$5"

  if [ "$enabled" != "1" ]; then
      return 0
  fi

  if [ ! -e "$real_dev" ]; then
      log "$label USB físico ausente: $real_dev"
      exit 0
  fi

  ensure_service_active "$svc"

  if [ ! -e "$virtual_dev" ]; then
      recover_pty "$svc" "$virtual_dev"
  fi

  if [ ! -e "$virtual_dev" ]; then
      log "$label PTY sigue ausente tras recuperación: $virtual_dev"
      exit 1
  fi
}
```

```
check_broker_alive() {
  if ! /usr/bin/systemctl is-active --quiet "$BROKER_SERVICE"; then
      log "Broker inactivo"
      restart_broker_now
  fi
}

check_direwolf_alive() {
  if [ "$APRS_GATE_ENABLED" = "1" ]; then
      if ! /usr/bin/systemctl is-active --quiet "$DIREWOLF_SERVICE"; then
          log "Direwolf inactivo. Reiniciando $DIREWOLF_SERVICE"
          /usr/bin/systemctl restart "$DIREWOLF_SERVICE"
          sleep 2
      fi
  fi
}

check_broker_zombie() {
  if [ ! -f "$STATUS_FILE" ]; then
      return 0
  fi

  local last_packet
  last_packet=$(/usr/bin/python3 - <<'PY' "$STATUS_FILE"
import json, sys
try:
  with open(sys.argv[1], "r", encoding="utf-8") as fh:
      data = json.load(fh)
  v = data.get("last_packet_ts")
  print("" if v is None else v)
except Exception:
  print("")
PY
)

  local now
  now=$(date +%s)

  if [ -n "$last_packet" ]; then
      local last_packet_int="${last_packet%.*}"
      case "$last_packet_int" in
          ''|*[!0-9]*) last_packet_int=0 ;;
      esac
      local diff=$((now - last_packet_int))
      if [ "$diff" -gt 60 ]; then
          log "Broker sin tráfico >60s (posible zombie)"
          restart_broker_now
      fi
```

```
  fi
}

check_meshcore_silence() {
  if [ ! -f "$STATUS_FILE" ]; then
    return 0
  fi

  local meshcore_ts
  meshcore_ts=$(/usr/bin/python3 - <<'PY' "$STATUS_FILE"
import json, sys
try:
  with open(sys.argv[1], "r", encoding="utf-8") as fh:
    data = json.load(fh)
  nodes = data.get("nodes") or []
  vals = []
  for node in nodes:
    if str(node.get("type") or "").startswith("meshcore"):
      v = node.get("last_packet_ts")
      if v is not None:
        vals.append(float(v))
  print("" if not vals else max(vals))
except Exception:
  print("")
PY
)

  local now
  now=$(date +%s)

  if [ -n "$meshcore_ts" ]; then
    local meshcore_int="${meshcore_ts%.*}"
    case "$meshcore_int" in
      "|*[!0-9]*) meshcore_int=0 ;;
    esac
    local diff=$((now - meshcore_int))
    if [ "$diff" -gt 180 ]; then
      log "MeshCore sin actividad >180s"
      restart_broker_now
    fi
  fi
}

check_usb_pair "$MESHTASTIC_ENABLE"   "$MESHTASTIC_USB_REAL"
"$MESHTASTIC_USB_VIRTUAL"   "$MESHTASTIC_SOCAT_SERVICE"   "Meshtastic-1"
check_usb_pair "$MESHTASTIC_ENABLE_2" "$MESHTASTIC_USB_REAL_2"
"$MESHTASTIC_USB_VIRTUAL_2" "$MESHTASTIC_SOCAT_SERVICE_2" "Meshtastic-2"
```

```
check_usb_pair "$MESHCORE_ENABLE"     "$MESHCORE_SERIAL_PORT"
"$MESHCORE_VIRTUAL_PORT"    "$MESHCORE_SOCAT_SERVICE"     "MeshCore-1"
check_usb_pair "$MESHCORE_ENABLE_2"   "$MESHCORE_SERIAL_PORT_2"
"$MESHCORE_VIRTUAL_PORT_2" "$MESHCORE_SOCAT_SERVICE_2"   "MeshCore-2"

if [ "$need_broker_restart" -eq 1 ]; then
  restart_broker_now
fi

check_broker_alive
check_direwolf_alive
check_broker_zombie

if [ "$MESHCORE_ENABLE" = "1" ] || [ "$MESHCORE_ENABLE_2" = "1" ]; then
  check_meshcore_silence
fi

log "OK: USB/PTYS y servicios activos"
exit 0
```

---

4. Servicio del watchdog

`/etc/systemd/system/minibroker-usb-watchdog.service`

```
[Unit]
Description=Watchdog MiniBroker 24x7

[Service]
Type=oneshot
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/usr/local/bin/minibroker_usb_watchdog.sh
```

`/etc/systemd/system/minibroker-usb-watchdog.timer`

```
[Unit]
Description=Timer watchdog MiniBroker

[Timer]
OnBootSec=45s
OnUnitActiveSec=60s
Unit=minibroker-usb-watchdog.service

[Install]
WantedBy=timers.target
```

---

5. Variantes correctas de `minibroker-emergencias.service`

El broker actual no debe quedarse acoplado solo a un Meshtastic USB legacy. El arranque bueno es por `.env-usb` multi-nodo.

5.1 Variante sin USB

Usar cuando todos los nodos sean TCP

```
[Unit]
Description=MiniBroker Emergencias 24x7 (sin USB)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=meshnet
Group=meshnet
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

5.2 Variante Meshtastic USB

```
[Unit]
Description=MiniBroker Emergencias 24x7 (Meshtastic USB)
After=network-online.target meshtastic-socat.service
Wants=network-online.target
Requires=meshtastic-socat.service

[Service]
Type=simple
User=meshnet
Group=meshnet
```

```
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

5.3 Variante MeshCore USB

```
[Unit]
Description=MiniBroker Emergencias 24x7 (MeshCore USB)
After=network-online.target meshcore-socat.service
Wants=network-online.target
Requires=meshcore-socat.service

[Service]
Type=simple
User=meshnet
Group=meshnet
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

5.4 Variante Meshtastic USB + MeshCore USB

```
[Unit]
Description=MiniBroker Emergencias 24x7 (Meshtastic USB + MeshCore USB)
After=network-online.target meshtastic-socat.service meshcore-socat.service
Wants=network-online.target
Requires=meshtastic-socat.service meshcore-socat.service
```

```
[Service]
Type=simple
User=meshnet
Group=meshnet
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

5.5 Variante 2× Meshtastic USB

```
[Unit]
Description=MiniBroker Emergencias 24x7 (2x Meshtastic USB)
After=network-online.target meshtastic-socat.service meshtastic-socat-2.service
Wants=network-online.target
Requires=meshtastic-socat.service meshtastic-socat-2.service

[Service]
Type=simple
User=meshnet
Group=meshnet
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

5.6 Variante 2× MeshCore USB

```
[Unit]
Description=MiniBroker Emergencias 24x7 (2x MeshCore USB)
After=network-online.target meshcore-socat.service meshcore-socat-2.service
Wants=network-online.target
Requires=meshcore-socat.service meshcore-socat-2.service

[Service]
Type=simple
User=meshnet
Group=meshnet
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

5.7 Variante 2× Meshtastic USB + 2× MeshCore USB

```
[Unit]
Description=MiniBroker Emergencias 24x7 (2x Meshtastic USB + 2x MeshCore USB)
After=network-online.target meshtastic-socat.service meshtastic-socat-2.service
meshcore-socat.service meshcore-socat-2.service
Wants=network-online.target
Requires=meshtastic-socat.service meshtastic-socat-2.service meshcore-socat.service
meshcore-socat-2.service

[Service]
Type=simple
User=meshnet
Group=meshnet
WorkingDirectory=/opt/minibroker
EnvironmentFile=/opt/minibroker/.env-usb
ExecStart=/opt/minibroker/venv/bin/python /opt/minibroker/emergency_broker.py --bind
127.0.0.1 --port 8765 --ctrl-host 127.0.0.1 --ctrl-port 8766 --data-dir /opt/minibroker/data
Restart=always
RestartSec=5
TimeoutStopSec=20
StartLimitBurst=0
NoNewPrivileges=true
PrivateTmp=true
```

```
[Install]
WantedBy=multi-user.target
```

---

6. Bloque base obligatorio para `.env-usb`

Este bloque debe existir siempre, aunque luego se activen o desactiven flags según cada escenario:

```
##########################################
# BROKER GLOBAL
##########################################
BROKER_HOST=127.0.0.1
BROKER_PORT=8765
BROKER_CTRL_HOST=127.0.0.1
BROKER_CTRL_PORT=8766
MINIBROKER_DATA_DIR=/opt/minibroker/data
BROKER_HEARTBEAT_SEC=15
BROKER_NO_HEARTBEAT=0

##########################################
# ENDURECIMIENTO USB/PTYS
##########################################

# Meshtastic USB 1
MESHTASTIC_ENABLE=0
MESHTASTIC_USB_REAL=/dev/ttyUSB0
MESHTASTIC_USB_VIRTUAL=/dev/ttyV0
MESHTASTIC_SOCAT_SERVICE=meshtastic-socat.service

# Meshtastic USB 2
MESHTASTIC_ENABLE_2=0
MESHTASTIC_USB_REAL_2=/dev/ttyUSB1
MESHTASTIC_USB_VIRTUAL_2=/dev/ttyV1
MESHTASTIC_SOCAT_SERVICE_2=meshtastic-socat-2.service

# MeshCore USB 1
MESHCORE_ENABLE=0
MESHCORE_SERIAL_PORT=/dev/ttyUSB2
MESHCORE_VIRTUAL_PORT=/dev/ttyMC0
MESHCORE_SOCAT_SERVICE=meshcore-socat.service

# MeshCore USB 2
```

```
MESHCORE_ENABLE_2=0
MESHCORE_SERIAL_PORT_2=/dev/ttyUSB3
MESHCORE_VIRTUAL_PORT_2=/dev/ttyMC1
MESHCORE_SOCAT_SERVICE_2=meshcore-socat-2.service

##########################################
# APRS
##########################################
APRS_GATE_ENABLED=0
KISS_HOST=127.0.0.1
KISS_PORT=8100
APRS_CALL=EA2XXX-10
APRS_PATH=WIDE1-1,WIDE2-1
APRSIS_USER=
APRSIS_PASSCODE=
APRSIS_HOST=rotate.aprs.net
APRSIS_PORT=14580
APRSIS_FILTER=
APRS_ALLOWED_SOURCES=
APRS_EMERGENCY_ALLOWED_CHANNELS=0
APRS_TX_MIN_INTERVAL_SEC=15
APRS_INBOUND_EMERGENCY_NO_CHTAG=1
MESHTASTIC_CH=0
APRS_INBOUND_NODE_ALIASES=
```

`direwolf.conf` ya está preparado con `KISSPORT 8100`, `MYCALL` y audio USB; con APRS activo solo hay que ajustar indicativo y tarjeta de audio.

---

7. Reglas rápidas para no equivocarse

Si un nodo es USB

- en `NODE_X_PORT` va el **PTY virtual**
- no va el USB real

Si un nodo es TCP

- se usa `NODE_X_HOST` y `NODE_X_TCP_PORT` o `NODE_X_MC_TCP_HOST` y `NODE_X_MC_TCP_PORT`

Si APRS está activo

- debe existir al menos un nodo Meshtastic
- `APRS_INBOUND_NODE_ALIASES` debe apuntar al alias del nodo o nodos que deban retransmitir la entrada APRS a la malla Mesh. El bridge permite alias múltiples.

Si el escenario es puramente MeshCore ↔ MeshCore

- APRS debe quedar desactivado

---

8. Escenarios completos

A continuación van todos los escenarios útiles y coherentes para dos nodos.

Escenario 1

Nodo A = Meshtastic USB + Nodo B = MeshCore TCP

Servicio del broker

- Variante: **Meshtastic USB**

Servicios `socat`

- `meshtastic-socat.service`

`.env-usb`

```
MESHTASTIC_ENABLE=1
MESHCORE_ENABLE=0

NODE_1_TYPE=meshtastic_serial
NODE_1_ALIAS=meshtastic-usb
NODE_1_PORT=/dev/ttyV0
NODE_1_BAUD=115200
NODE_1_PRIMARY=1

NODE_2_TYPE=meshcore_tcp
NODE_2_ALIAS=meshcore-tcp
NODE_2_MC_TCP_HOST=192.168.1.60
NODE_2_MC_TCP_PORT=4000
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC
```

NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshtastic-usb:0,dst:meshcore-tcp:0
NODE_ROUTE_2=src:meshcore-tcp:0,dst:meshtastic-usb:0

Sin APRS

APRS_GATE_ENABLED=0
APRS_INBOUND_NODE_ALIASES=

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb
MESHTASTIC_CH=0

Escenario 2

Nodo A = Meshtastic TCP + Nodo B = MeshCore TCP

Servicio del broker

- Variante: **sin USB**

Servicios `socat`

- ninguno

`.env-usb`

MESHTASTIC_ENABLE=0
MESHCORE_ENABLE=0

NODE_1_TYPE=meshtastic_tcp
NODE_1_ALIAS=meshtastic-tcp
NODE_1_HOST=192.168.1.50
NODE_1_TCP_PORT=4403
NODE_1_PRIMARY=1

NODE_2_TYPE=meshcore_tcp
NODE_2_ALIAS=meshcore-tcp
NODE_2_MC_TCP_HOST=192.168.1.60
NODE_2_MC_TCP_PORT=4000
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC

NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshtastic-tcp:0,dst:meshcore-tcp:0
NODE_ROUTE_2=src:meshcore-tcp:0,dst:meshtastic-tcp:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-tcp
MESHTASTIC_CH=0

Escenario 3

Nodo A = Meshtastic USB + Nodo B = MeshCore USB

Servicio del broker

- Variante: **Meshtastic USB + MeshCore USB**

Servicios `socat`

- `meshtastic-socat.service`
- `meshcore-socat.service`

`.env-usb`

MESHTASTIC_ENABLE=1
MESHCORE_ENABLE=1

NODE_1_TYPE=meshtastic_serial
NODE_1_ALIAS=meshtastic-usb
NODE_1_PORT=/dev/ttyV0
NODE_1_BAUD=115200
NODE_1_PRIMARY=1

NODE_2_TYPE=meshcore_serial
NODE_2_ALIAS=meshcore-usb
NODE_2_PORT=/dev/ttyMC0
NODE_2_BAUD=115200
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC

NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshtastic-usb:0,dst:meshcore-usb:0
NODE_ROUTE_2=src:meshcore-usb:0,dst:meshtastic-usb:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb
MESHTASTIC_CH=0

Escenario 4

Nodo A = MeshCore USB + Nodo B = Meshtastic TCP

Servicio del broker

- Variante: **MeshCore USB**

Servicios `socat`

- `meshcore-socat.service`

`.env-usb`

MESHTASTIC_ENABLE=0
MESHCORE_ENABLE=1

NODE_1_TYPE=meshcore_serial
NODE_1_ALIAS=meshcore-usb
NODE_1_PORT=/dev/ttyMC0
NODE_1_BAUD=115200
NODE_1_PRIMARY=1
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshtastic_tcp
NODE_2_ALIAS=meshtastic-tcp

NODE_2_HOST=192.168.1.50
NODE_2_TCP_PORT=4403

NODE_ROUTE_1=src:meshcore-usb:0,dst:meshtastic-tcp:0
NODE_ROUTE_2=src:meshtastic-tcp:0,dst:meshcore-usb:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-tcp
MESHTASTIC_CH=0

Escenario 5

Nodo A = MeshCore TCP + Nodo B = Meshtastic TCP

Servicio del broker

- Variante: **sin USB**

Servicios `socat`

- ninguno

`.env-usb`

MESHTASTIC_ENABLE=0
MESHCORE_ENABLE=0

NODE_1_TYPE=meshcore_tcp
NODE_1_ALIAS=meshcore-tcp
NODE_1_MC_TCP_HOST=192.168.1.60
NODE_1_MC_TCP_PORT=4000
NODE_1_PRIMARY=1
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshtastic_tcp
NODE_2_ALIAS=meshtastic-tcp
NODE_2_HOST=192.168.1.50

NODE_2_TCP_PORT=4403

NODE_ROUTE_1=src:meshcore-tcp:0,dst:meshtastic-tcp:0
NODE_ROUTE_2=src:meshtastic-tcp:0,dst:meshcore-tcp:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-tcp
MESHTASTIC_CH=0

Escenario 6

Nodo A = MeshCore USB + Nodo B = Meshtastic USB

Servicio del broker

- Variante: **Meshtastic USB + MeshCore USB**

Servicios `socat`

- `meshcore-socat.service`
- `meshtastic-socat.service`

`.env-usb`

MESHTASTIC_ENABLE=1
MESHCORE_ENABLE=1

NODE_1_TYPE=meshcore_serial
NODE_1_ALIAS=meshcore-usb
NODE_1_PORT=/dev/ttyMC0
NODE_1_BAUD=115200
NODE_1_PRIMARY=1
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshtastic_serial
NODE_2_ALIAS=meshtastic-usb
NODE_2_PORT=/dev/ttyV0

NODE_2_BAUD=115200

NODE_ROUTE_1=src:meshcore-usb:0,dst:meshtastic-usb:0
NODE_ROUTE_2=src:meshtastic-usb:0,dst:meshcore-usb:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb
MESHTASTIC_CH=0

Escenario 7

Nodo A = Meshtastic USB + Nodo B = Meshtastic TCP

Servicio del broker

- Variante: **Meshtastic USB**

Servicios `socat`

- `meshtastic-socat.service`

`.env-usb`

MESHTASTIC_ENABLE=1

NODE_1_TYPE=meshtastic_serial
NODE_1_ALIAS=meshtastic-usb
NODE_1_PORT=/dev/ttyV0
NODE_1_PRIMARY=1

NODE_2_TYPE=meshtastic_tcp
NODE_2_ALIAS=meshtastic-tcp
NODE_2_HOST=192.168.1.50
NODE_2_TCP_PORT=4403

NODE_ROUTE_1=src:meshtastic-usb:0,dst:meshtastic-tcp:0
NODE_ROUTE_2=src:meshtastic-tcp:0,dst:meshtastic-usb:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb
MESHTASTIC_CH=0

Escenario 8

Nodo A = Meshtastic TCP + Nodo B = Meshtastic USB

Servicio del broker

- Variante: **Meshtastic USB**

Servicios `socat`

- `meshtastic-socat.service`

`.env-usb`

MESHTASTIC_ENABLE=1

NODE_1_TYPE=meshtastic_tcp
NODE_1_ALIAS=meshtastic-tcp
NODE_1_HOST=192.168.1.50
NODE_1_TCP_PORT=4403
NODE_1_PRIMARY=1

NODE_2_TYPE=meshtastic_serial
NODE_2_ALIAS=meshtastic-usb
NODE_2_PORT=/dev/ttyV0

NODE_ROUTE_1=src:meshtastic-tcp:0,dst:meshtastic-usb:0
NODE_ROUTE_2=src:meshtastic-usb:0,dst:meshtastic-tcp:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb
MESHTASTIC_CH=0

Escenario 9

Nodo A = Meshtastic USB + Nodo B = Meshtastic USB

Servicio del broker

- Variante: **2× Meshtastic USB**

Servicios `socat`

- `meshtastic-socat.service`
- `meshtastic-socat-2.service`

`.env-usb`

```
MESHTASTIC_ENABLE=1
MESHTASTIC_ENABLE_2=1

NODE_1_TYPE=meshtastic_serial
NODE_1_ALIAS=meshtastic-usb-1
NODE_1_PORT=/dev/ttyV0
NODE_1_PRIMARY=1

NODE_2_TYPE=meshtastic_serial
NODE_2_ALIAS=meshtastic-usb-2
NODE_2_PORT=/dev/ttyV1

NODE_ROUTE_1=src:meshtastic-usb-1:0,dst:meshtastic-usb-2:0
NODE_ROUTE_2=src:meshtastic-usb-2:0,dst:meshtastic-usb-1:0
```

Sin APRS

```
APRS_GATE_ENABLED=0
```

Con APRS

```
APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb-1
MESHTASTIC_CH=0
```

Escenario 10

Nodo A = Meshtastic TCP + Nodo B = Meshtastic TCP

Servicio del broker

- Variante: **sin USB**

Servicios `socat`

- ninguno

`.env-usb`

```
NODE_1_TYPE=meshtastic_tcp
NODE_1_ALIAS=meshtastic-tcp-1
NODE_1_HOST=192.168.1.50
NODE_1_TCP_PORT=4403
NODE_1_PRIMARY=1

NODE_2_TYPE=meshtastic_tcp
NODE_2_ALIAS=meshtastic-tcp-2
NODE_2_HOST=192.168.1.51
NODE_2_TCP_PORT=4403

NODE_ROUTE_1=src:meshtastic-tcp-1:0,dst:meshtastic-tcp-2:0
NODE_ROUTE_2=src:meshtastic-tcp-2:0,dst:meshtastic-tcp-1:0
```

Sin APRS

```
APRS_GATE_ENABLED=0
```

Con APRS

```
APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-tcp-1
MESHTASTIC_CH=0
```

Escenario 11

Nodo A = MeshCore USB + Nodo B = MeshCore TCP

Servicio del broker

- Variante: **MeshCore USB**

Servicios `socat`

- `meshcore-socat.service`

`.env-usb`

```
MESHCORE_ENABLE=1
```

NODE_1_TYPE=meshcore_serial
NODE_1_ALIAS=meshcore-usb
NODE_1_PORT=/dev/ttyMC0
NODE_1_PRIMARY=1
NODE_1_BAUD=115200
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshcore_tcp
NODE_2_ALIAS=meshcore-tcp
NODE_2_MC_TCP_HOST=192.168.1.60
NODE_2_MC_TCP_PORT=4000
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshcore-usb:0,dst:meshcore-tcp:0
NODE_ROUTE_2=src:meshcore-tcp:0,dst:meshcore-usb:0

APRS

- **Siempre desactivado**

APRS_GATE_ENABLED=0

Escenario 12

Nodo A = MeshCore TCP + Nodo B = MeshCore USB

Servicio del broker

- Variante: **MeshCore USB**

Servicios `socat`

- `meshcore-socat.service`

`.env-usb`

MESHCORE_ENABLE=1

NODE_1_TYPE=meshcore_tcp
NODE_1_ALIAS=meshcore-tcp
NODE_1_MC_TCP_HOST=192.168.1.60
NODE_1_MC_TCP_PORT=4000
NODE_1_PRIMARY=1
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshcore_serial
NODE_2_ALIAS=meshcore-usb
NODE_2_PORT=/dev/ttyMC0
NODE_2_BAUD=115200
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshcore-tcp:0,dst:meshcore-usb:0
NODE_ROUTE_2=src:meshcore-usb:0,dst:meshcore-tcp:0

APRS

APRS_GATE_ENABLED=0

Escenario 13

Nodo A = MeshCore USB + Nodo B = MeshCore USB

Servicio del broker

- Variante: **2× MeshCore USB**

Servicios `socat`

- `meshcore-socat.service`
- `meshcore-socat-2.service`

`.env-usb`

MESHCORE_ENABLE=1
MESHCORE_ENABLE_2=1

NODE_1_TYPE=meshcore_serial
NODE_1_ALIAS=meshcore-usb-1

```
NODE_1_PORT=/dev/ttyMC0
NODE_1_PRIMARY=1
NODE_1_BAUD=115200
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshcore_serial
NODE_2_ALIAS=meshcore-usb-2
NODE_2_PORT=/dev/ttyMC1
NODE_2_BAUD=115200
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshcore-usb-1:0,dst:meshcore-usb-2:0
NODE_ROUTE_2=src:meshcore-usb-2:0,dst:meshcore-usb-1:0
```

APRS

```
APRS_GATE_ENABLED=0
```

Escenario 14

Nodo A = MeshCore TCP + Nodo B = MeshCore TCP

Servicio del broker

- Variante: **sin USB**

Servicios `socat`

- ninguno

`.env-usb`

```
NODE_1_TYPE=meshcore_tcp
NODE_1_ALIAS=meshcore-tcp-1
NODE_1_MC_TCP_HOST=192.168.1.60
NODE_1_MC_TCP_PORT=4000
NODE_1_PRIMARY=1
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
```

NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshcore_tcp
NODE_2_ALIAS=meshcore-tcp-2
NODE_2_MC_TCP_HOST=192.168.1.61
NODE_2_MC_TCP_PORT=4000
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_2_MC_CHANIDX_TO_CH=0:0
NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshcore-tcp-1:0,dst:meshcore-tcp-2:0
NODE_ROUTE_2=src:meshcore-tcp-2:0,dst:meshcore-tcp-1:0

APRS

APRS_GATE_ENABLED=0

Escenario 15

Nodo A = Meshtastic TCP + Nodo B = MeshCore USB

Servicio del broker

- Variante: **MeshCore USB**

Servicios `socat`

- `meshcore-socat.service`

`.env-usb`

MESHCORE_ENABLE=1

NODE_1_TYPE=meshtastic_tcp
NODE_1_ALIAS=meshtastic-tcp
NODE_1_HOST=192.168.1.50
NODE_1_TCP_PORT=4403
NODE_1_PRIMARY=1

NODE_2_TYPE=meshcore_serial
NODE_2_ALIAS=meshcore-usb
NODE_2_PORT=/dev/ttyMC0
NODE_2_BAUD=115200
NODE_2_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_2_MC_CHANIDX_TO_CH=0:0

NODE_2_MC_RX_PREFIX_STYLE=alias
NODE_2_MC_DEFAULT_CH=0

NODE_ROUTE_1=src:meshtastic-tcp:0,dst:meshcore-usb:0
NODE_ROUTE_2=src:meshcore-usb:0,dst:meshtastic-tcp:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-tcp
MESHTASTIC_CH=0

Escenario 16

Nodo A = MeshCore TCP + Nodo B = Meshtastic USB

Servicio del broker

- Variante: **Meshtastic USB**

Servicios `socat`

- `meshtastic-socat.service`

`.env-usb`

MESHTASTIC_ENABLE=1

NODE_1_TYPE=meshcore_tcp
NODE_1_ALIAS=meshcore-tcp
NODE_1_MC_TCP_HOST=192.168.1.60
NODE_1_MC_TCP_PORT=4000
NODE_1_PRIMARY=1
NODE_1_MC_CHANNEL_MAP=0:chan:0:PUBLIC
NODE_1_MC_CHANIDX_TO_CH=0:0
NODE_1_MC_RX_PREFIX_STYLE=alias
NODE_1_MC_DEFAULT_CH=0

NODE_2_TYPE=meshtastic_serial
NODE_2_ALIAS=meshtastic-usb
NODE_2_PORT=/dev/ttyV0

NODE_ROUTE_1=src:meshcore-tcp:0,dst:meshtastic-usb:0
NODE_ROUTE_2=src:meshtastic-usb:0,dst:meshcore-tcp:0

Sin APRS

APRS_GATE_ENABLED=0

Con APRS

APRS_GATE_ENABLED=1
APRS_INBOUND_NODE_ALIASES=meshtastic-usb
MESHTASTIC_CH=0

---

9. Qué debe activar la gente en cada escenario

Escenarios con Meshtastic USB

Activar:

- `meshtastic-socat.service`
- `MESHTASTIC_ENABLE=1`
- `NODE_X_PORT=/dev/ttyV0` o `/dev/ttyV1`

Escenarios con MeshCore USB

Activar:

- `meshcore-socat.service`
- `MESHCORE_ENABLE=1`
- `NODE_X_PORT=/dev/ttyMC0` o `/dev/ttyMC1`

Escenarios solo TCP

- no activar ningún `socat`
- usar broker variante **sin USB**

Escenarios con APRS

Activar:

- `APRS_GATE_ENABLED=1`

- `direwolf.service`
- `minibroker-aprs.service`

Y definir correctamente:

- `APRS_CALL`
- `KISS_HOST=127.0.0.1`
- `KISS_PORT=8100`
- `APRS_INBOUND_NODE_ALIASES=<alias_meshtastic>`
- `MESHTASTIC_CH=<canal>`

Dire Wolf debe tener `KISSPORT 8100`, `MYCALL` y `ADEVICE` válidos.

---

10. Resumen operativo final

Si quieres el diseño correcto y estable 24x7

- Meshtastic USB y MeshCore USB deben ir ambos por **PTY virtual**
- el broker no debe abrir directamente los USB reales
- el watchdog debe ser **único y genérico**
- APRS solo se activa cuando hay al menos un nodo Meshtastic
- `APRS_INBOUND_NODE_ALIASES` debe apuntar al alias real del nodo Meshtastic que va a radiar la entrada APRS; el bridge lo soporta expresamente.

Escenarios puros MeshCore

- APRS apagado
- sin Direwolf
- sin `minibroker-aprs.service`

Escenarios mixtos Meshtastic + MeshCore

- routing normal entre nodos por `NODE_ROUTE_N`
- APRS, si se activa, entra y sale por el/los alias Meshtastic configurados

Si quieres el siguiente paso, lo correcto es convertir esto ya en una **guía de implantación final en Markdown**, con bloques listos para copiar por escenario y una tabla de “qué habilitar / qué deshabilitar” para que cualquier usuario lo despliegue sin pensar.